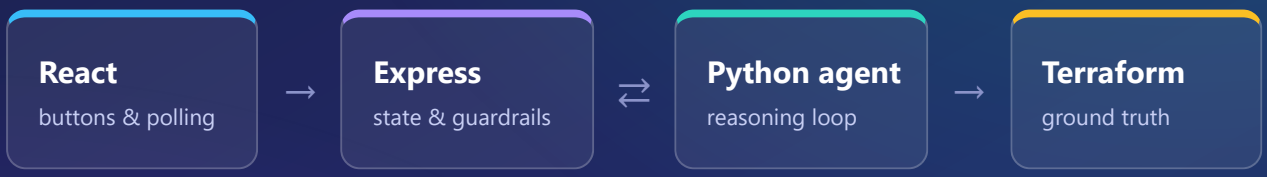
INFRAFLOW · KNOWLEDGE TRANSFER

The AI Agent, start to finish

How to read the agent code in order — from the browser click to a real `terraform plan` — with a deep dive on the "make it pass terraform plan" tool-calling loop.



0 The mental model

1 Reading order — 27 files, 7 phases

2 One plan-fix run, end to end

3 Design rules

4 Dependency injection in the loop

5 Hands-on exercises

6 Known gaps

7 Local setup reference

0 The mental model

Three processes. Each owns exactly one thing — and is forbidden from the rest.

React
src/

OWNS

Buttons and polling.

NEVER

Talks to the Python service.

Express
IAAS backend/

OWNS

All state (Mongo), AWS credentials, running `terraform`, **every guardrail**.

NEVER

Decides what to fix.

Python agent
services/agents/

OWNS

The LLM's reasoning and its loop.

NEVER

Holds an AWS SDK, a DB driver or a terraform binary — only a short-lived token, calling Express for everything.

READ FIRST

chat — fixed workflow

A LangGraph graph. **The code decides the steps**: route, fan out to specialists, synthesize, propose a ChangePlan. One invocation per message.

BUILDS ON CHAT

plan_fix — open-ended loop

A tool-calling loop. **The model decides the steps**: edit a draft, read HCL, run real plans, finish. Express enforces every rule around it.

1 Reading order

27 files in seven phases — roughly one focused day. Paths are relative to the repo root.

★ **Short on time? Read the four fast-track files.**

#3 AgentRun.js → **#10** runner.py → **#17** planfix/agent.py → **#18** agentPlanFix.js. Together they hold about 80% of the agent design.

A Orientation — 15 min

1 `services/agents/README.md`

The *Two kinds of run* section is the whole design in two paragraphs.

2 `services/agents/app/main.py`

The header comment walks through the 4-step flow.

3 `IAAS backend/src/models/AgentRun.js` ★ FAST TRACK

The most important data structure. One Mongo document per run: `kind`, `status`, `steps`, and `fix` — the plan-fix state that survives between invocations.

B How a run starts & how auth works — 30 min

4 `IAAS backend/src/routes/agentRoutes.js`

User-facing routes, behind `requireAuth`.

5 `IAAS backend/src/routes/internalAgentTools.js`

Routes the **Python service** calls back into. Each one requires a scope.

6 `IAAS backend/src/services/agentIdentity.js`

Mints a 5-minute machine JWT per run.

7 `IAAS backend/src/utils/agentBridge.js`

`startAgentRun()` : two scope sets (`RUN_SCOPES` , `PLAN_FIX_SCOPES`) and two transports — HTTP locally, async Lambda `Event` invoke in production.

8 `IAAS backend/src/middleware/requireAgentScope.js` `services/agents/app/security.py`

The same JWT, verified on both sides with the shared `AGENT_TOOL_SECRET` .

C The Python service skeleton — 30 min

9 `services/agents/app/api/routes_runs.py` `services/agents/app/lambda_handler.py`

uvicorn and Lambda entry points. Both branch on `mode == "plan_fix"` .

10 `services/agents/app/runner.py` ★ FAST TRACK

Why runs are fire-and-forget on uvicorn but awaited on Lambda (Lambda freezes once the handler returns). In `_run_plan_fix` : the wall-clock timeout, and a `409` meaning the run is already closed.

11 `services/agents/app/gateway.py`

The service's only window to the world. Every capability is an `httpx` POST to Express. The four `fix_*` tool wrappers are at the bottom.

D The chat graph (warm-up) — 45 min

12 `services/agents/app/graph.py`

`build_graph()` : ground → supervisor → `Send` [specialists in parallel] → synthesize → changeplan → END.

13 `services/agents/app/state.py`

`Annotated[list, add]` lets parallel specialists append to `results` instead of overwriting each other.

14 `services/agents/app/agents/base.py`

`call_llm` (one-shot JSON), `call_llm_with_tools` (**the primitive plan-fix is built on**), and retry with exponential backoff.

15 `services/agents/app/dsl.py`

How the diagram becomes compact prompt text, with node IDs in `[brackets]`.

16 `services/agents/app/evidence.py` `services/agents/app/changeplan.py`

Mechanical gates. The model can say anything; only claims citing real node IDs or Terraform addresses survive.

E The plan-fix agent — the core

2 hrs

17 `services/agents/app/planfix/agent.py` ★ FAST TRACK

The brain. Read in this order:

- `SYSTEM_PROMPT` and `TOOLS` — the five tools: `apply_changes`, `get_terraform_block`, `view_draft`, `run_terraform_plan`, `finish`
- `PlanFixSession.run()` (line 166) — **the agent loop**: call the LLM, post its note as `fix.note`, run ordinary tools, then at most one terminal tool
- `_run_tool` / `_run_terminal` — each tool is one `gateway.fix_*` call; only `get_terraform_block` runs locally (`planfix/hcl.py`)
- `compact_tool_results` keeps context bounded; `plan_result_payload` feeds a plan result back as a tool response

18 `IAAS backend/src/services/agentPlanFix.js` ★ FAST TRACK

The guardrails. Map each function to the tool it serves:

- `createPlanFixState` — the draft is a `structuredClone` of the diagram, revision 0
- `applyOneChange` — rejects `remove_node`, secret-looking keys (`SENSITIVE_KEY`), unknown IDs and service IDs
- `recordAgentChanges` — edit budget; bumps `revision` when anything applies
- `requestPlanRun` — plan budget, no re-plan of an unchanged revision, **saves messages + `pendingToolCallId`**, starts the plan
- `finishPlanFix` — the gate: *passed* only if the latest plan passed on the **current** revision
- `completePlanRun` → `resumePlanFixAgent` — store the result, start a **new** invocation
- `normalizePlanDiagnostics` — Terraform error → resource address → diagram node ID

19 `IAAS backend/src/controllers/internalAgentToolsController.js`

Thin HTTP wrappers (bottom section). `loadOwnFixRun`: your own run only, and only while it is running.

20 `.github/workflows/terraform-plan.yml`

The production plan executor. Its last step **always** calls back so the paused agent resumes.

21 `IAAS backend/src/controllers/agentController.js`

`startPlanFix`, `receivePlanCallback`, and `applyPlanFix` — the *only* path from draft to real diagram, triggered by a user click.

F Frontend — 30 min

22 `src/dashboard/agentApi.ts`

The types are the cleanest description of the API contract: `AgentStepEvent`, `PlanFixView`, `AgentRunStatus`.

23 `src/dashboard/pages/AgentPage.tsx`

`startPlanFixRun` → `startPolling` → `getAgentRunStatus` on an interval → `handleApplyFix`.

24 `src/dashboard/components/AgentPlanFixPanel.tsx` `AgentRunTimeline.tsx`

Render each `step.type` as a live timeline, plus the Apply button.

G Tests as executable docs — 20 min

25 `services/agents/tests/test_planfix_agent.py`

Drives the loop with a scripted fake LLM and a fake Express. **The best way to watch the loop move.**

26 `IAAS backend/test/agentPlanFix.test.js`

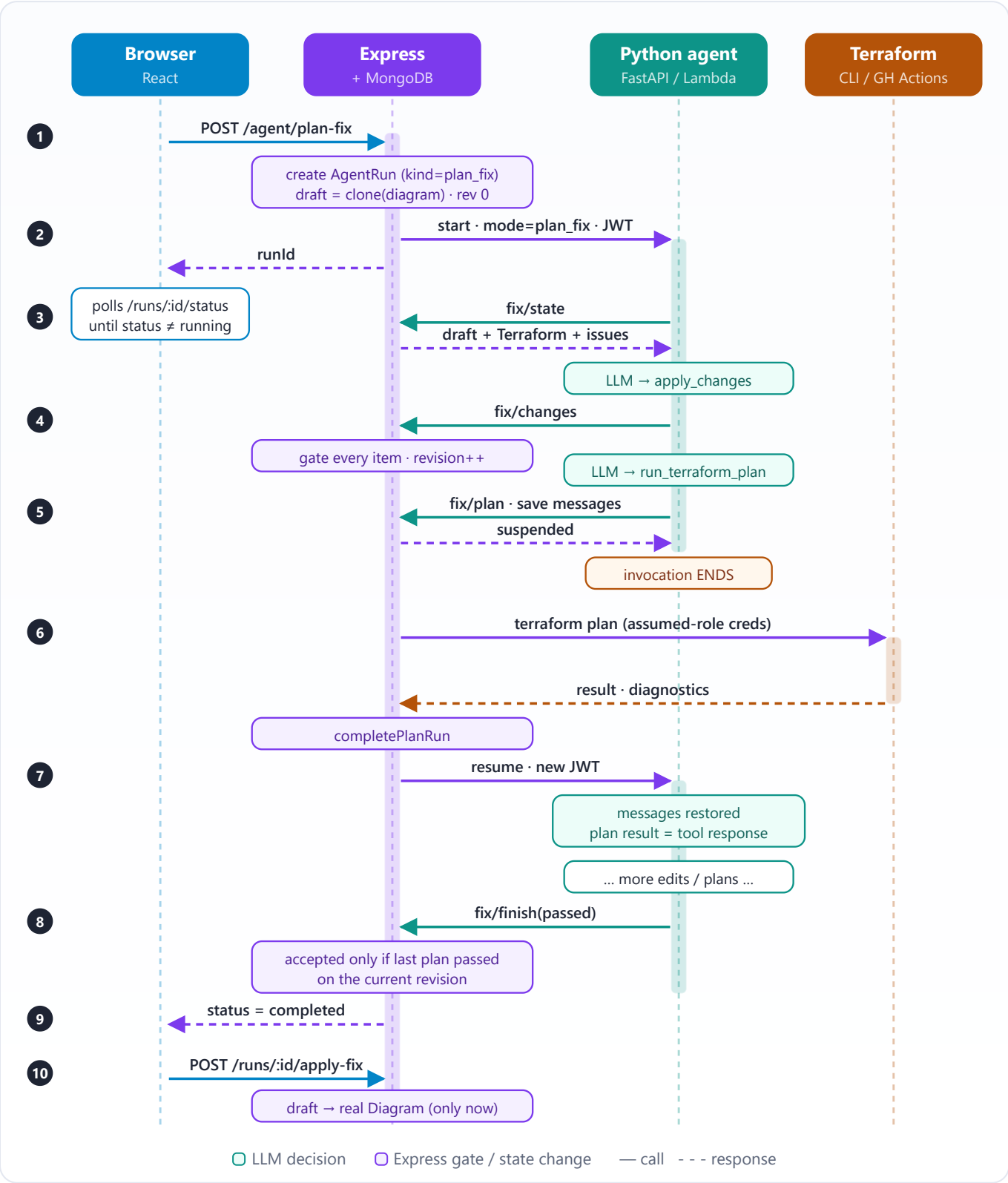
The guardrails, as tests.

27 `services/agents/tests/test_no_cloud_sdk.py`

Enforces “the Python service has no cloud SDK” as a test rather than a comment.

2 One plan-fix run, end to end

From the click to the applied diagram. Shaded bars show when each process is actually working.



> **The key insight: one run, many invocations**
>
> The agent's whole conversation lives in Mongo (`fix.agent.messages`). While a plan runs, **no agent process exists at all**. When the plan finishes, Express mints a fresh token and starts a new invocation, and the plan result is attached to the tool call the agent was waiting on (`pendingToolCallId`). That is how a 1–3 minute `terraform plan` fits inside a 120 s Lambda.

Step events — what lands in `AgentRun.steps[]`

Step type	Written by	Meaning
CHAT RUNS		
`grounding.ready`	runner.py	Diagram bundle fetched
`route.decided`	runner.py	Supervisor picked the specialists
`agent.finished`	runner.py	One specialist answered
`changeplan.ready`	runner.py	Proposed ChangePlan is ready
PLAN_FIX RUNS		
`fix.started`	agentController.startPlanFix	Plan-fix run created
`fix.note`	planfix/agent.py	The model's live reasoning note before a tool call
`fix.changes`	agentPlanFix.recordAgentChanges	An edit batch was applied or rejected
`plan.started` / `plan.finished`	requestPlanRun / completePlanRun	A real `terraform plan` started or finished
`fix.finished` / `fix.failed`	finishPlanFix / failPlanFixRun	Terminal state

3 Design rules

Recognise these as you read — they explain almost every "why is it done this way?".

The model proposes; code enforces 1

Every rule in `SYSTEM_PROMPT` is *also* enforced in `agentPlanFix.js`. The prompt makes the agent better behaved — safety comes from code.

Draft, not the real diagram 2

The token carries `diagram:draft:write` and nothing that writes a real diagram. Only `applyPlanFix` — a user click — writes, and it refuses if the diagram changed since the run started.

Budgets at three levels 3

- Per invocation: `SOFT_DEADLINE_SECONDS` (85 s)
- Hard wall: `PLAN_FIX_INVOCATION_WALL_SECONDS` (105 s)
- Per run: `PLAN_FIX_LIMITS` — 5 plans, 40 edits, 40 turns

Every run reaches a terminal state 4

Look for `except Exception: … _fail_plan_fix`. A run must never sit at `running` forever.

Rejections are tool results, not crashes 5

A bad edit returns `{rejected: [...reason]}` so the model can read the reason and correct itself.

Untrusted data stays data 6

Diagram, Terraform and plan output are wrapped and labelled as data. The model is told never to follow instructions inside them.

4 Dependency injection in the loop

services/agents/app/planfix/agent.py, line 144

```
LlmCall = Callable[[list[dict], list[dict], str | None], Awaitable[object]]

class PlanFixSession:
    def __init__(self, run_id, token, state, *,
                 gateway=default_gateway,
                 llm: LlmCall = call_llm_with_tools, ...)
```

"An async function that takes `(messages, tools, model)` and returns the assistant message" — the exact signature of `call_llm_with_tools`. Because `PlanFixSession` receives `llm` and `gateway` as arguments instead of calling OpenAI or Express directly, production uses the real implementations while `test_planfix_agent.py` passes a scripted fake model and an in-memory Express — so the whole loop runs in tests with no network access.

5 Hands-on exercises

Reading gets you the map; these get you the territory.

1. **Run the Python tests with output.** `pytest tests/test_planfix_agent.py -v -s` from `services/agents`, then add `print(self.messages[-1])` inside the `run()` loop and watch the conversation grow.
2. **Set breakpoints across the boundary** at `PlanFixSession.run` (Python) and `requestPlanRun` / `completePlanRun` (Node), then click "Make it pass terraform plan" locally.
3. **Watch the AgentRun document in Mongo** during a run: `steps[]` grows, `fix.revision` climbs, and `fix.agent.messages` is the raw transcript.
4. **Try to break a guardrail.** Write a test where the fake LLM calls `finish(passed)` before any plan, or sends `remove_node`. Confirm Express rejects it and the agent receives the reason.
5. **Explain it back.** What happens if the GitHub Action never calls back? (Answer below.)

6 Known gaps

> **!** **Lost plan callback**
>
> If `terraform-plan.yml` never reports back (runner outage, deleted workflow), the run stays `running` in Mongo. The UI stops polling at its own deadline (`PLAN_FIX_MAX_POLL_MS` in `AgentPage.tsx`), but nothing closes the run on the server. A periodic job that fails plan-fix runs whose latest plan has been `running` too long would close this gap.

7 Local setup reference

Env vars and commands for running the agent locally.

What	Where
Agent service env	`services/agents/.env` — `OPENAI_API_KEY` , `OPENAI_FIX_MODEL` (optional), `AGENT_TOOL_SECRET` , `EXPRESS_INTERNAL_URL`
Backend env	`IAAS backend/.env` — `AGENT_TOOL_SECRET` (must match), `AGENT_API_URL` , `AGENT_INVOKE_MODE` , `DEPLOYMENT_EXECUTOR` , `TERRAFORM_BIN`
Run the agent service	`.venv\Scripts\uvicorn app.main:app --port 8001 --reload` (from `services/agents`)
Production deploy	`services/agents/deploy/README.md`
Regenerate AWS schema	`npm run gen:agent-schemas` (repo root)